Presented by :
Cielo Escobar

Instrumented & Monitored Cloud Service

This project focuses on building a cloud service that is fully observable.

The system includes:

- Structured logging
- Custom application metrics
- Monitoring dashboards
- Alerting system
- Incident investigation

The goal is to make the system easy to monitor, debug, and operate in production.

02

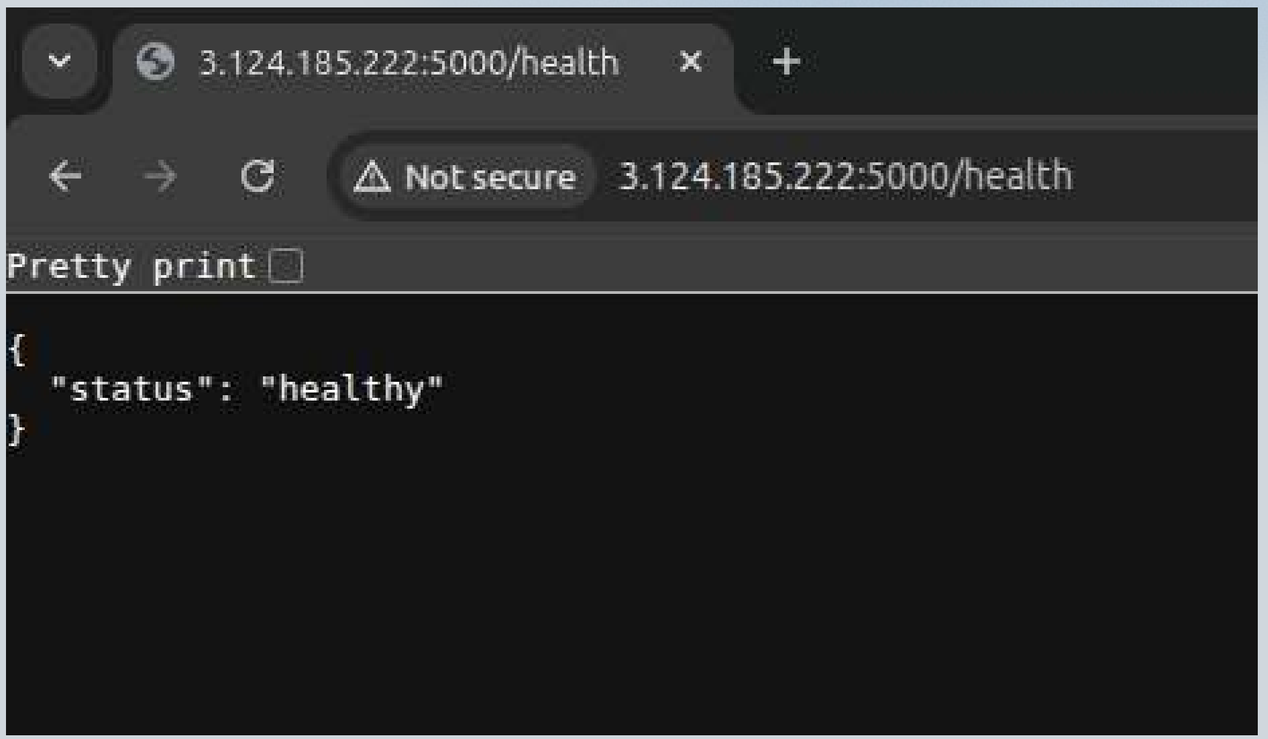
System Architecture

The application was deployed on AWS EC2 and instrumented with CloudWatch.

Architecture components:

Client Request
⬇
EC2 Instance
⬇
Flask Application
⬇
CloudWatch Observability Stack

Observability tools used:
- CloudWatch Logs for application logs
- CloudWatch Metrics for performance metrics
- CloudWatch Dashboards for visualization
- CloudWatch Alarms for monitoring
- SNS for alert notifications



03

Application Deployment

The application is implemented as a Flask web service and deployed on an AWS EC2 instance.

The service exposes API endpoints that allow clients to interact with the system.

Core endpoints:
GET /health — verifies service health
POST /orders — creates a new order

This deployment forms the base for implementing observability features such as logging, metrics, and monitoring.

```
(venv) ubuntu@ip-172-31-1-96:~/app$ python3 server.py
 * Serving Flask app 'server'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.31.1.96:5000
Press CTRL+C to quit
```

04

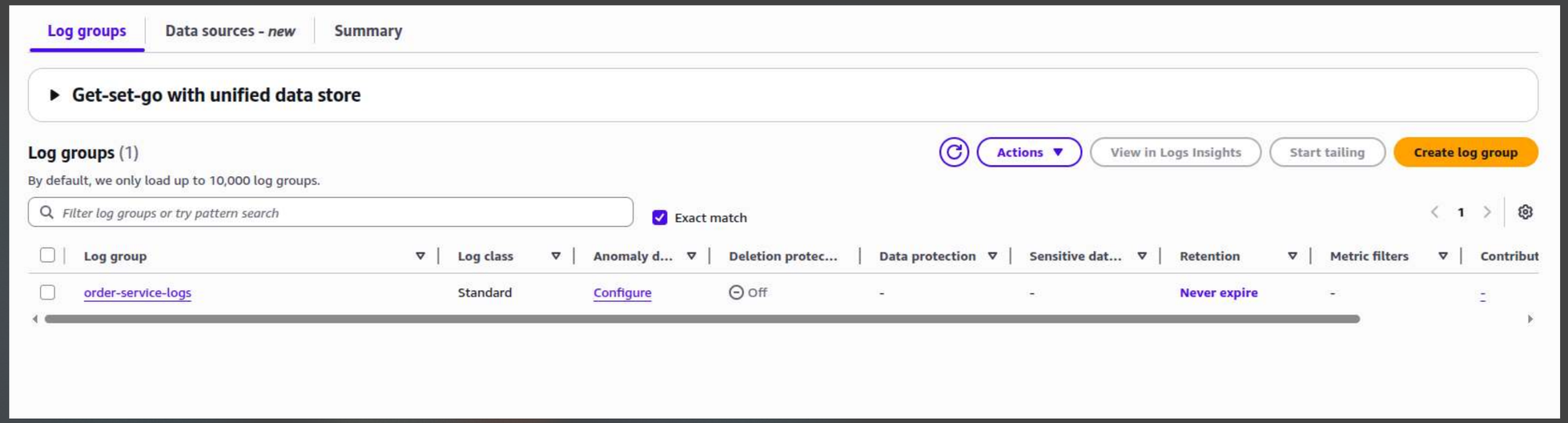
Structured Logging

The application uses structured JSON logging to improve observability and troubleshooting.

Structured logs include key fields such as:

- timestamp
- log level
- event name
- request metadata

Sending logs to CloudWatch allows centralized log storage and enables powerful search and analysis capabilities.



05

Log Analysis

Centralized logging enables engineers to analyze individual events generated by the application.

By inspecting log events, engineers can trace requests, identify errors, and understand the sequence of operations performed by the system.

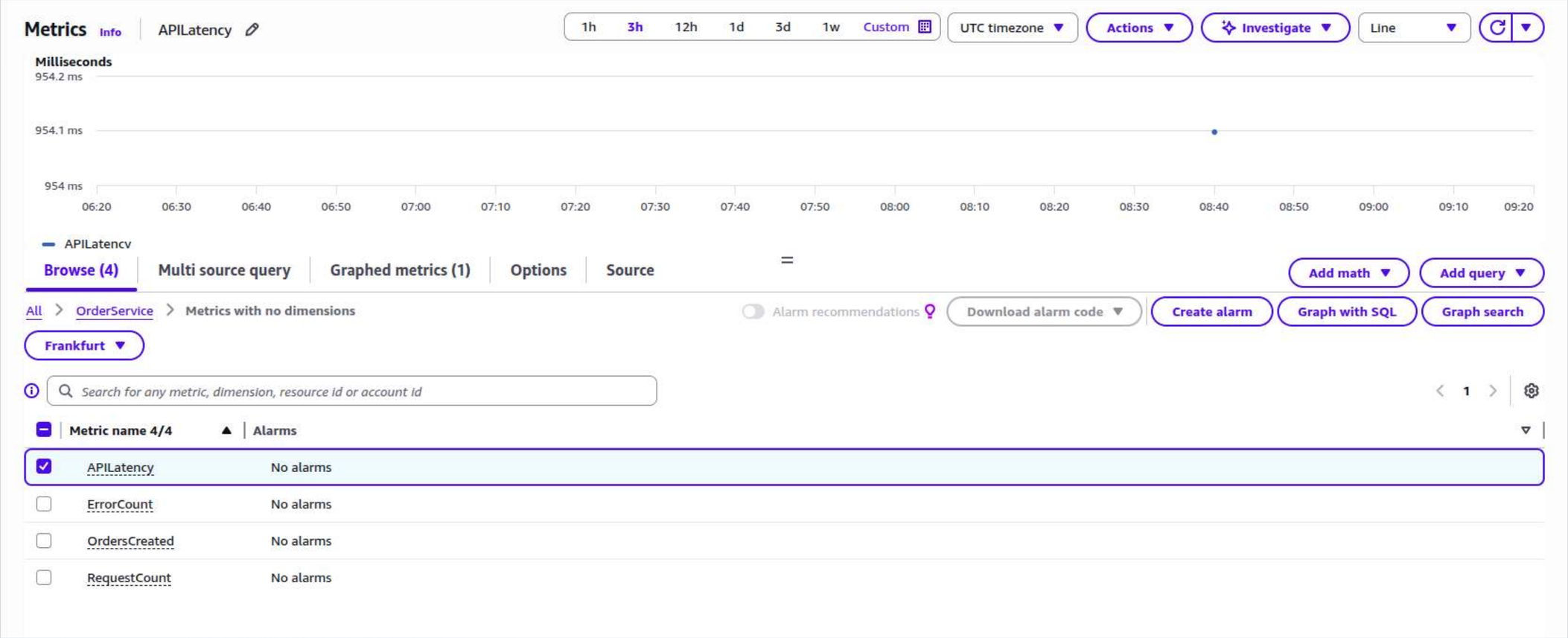
Log analysis is especially useful during debugging and incident investigation because it provides detailed insight into application activity.

06

Custom Metrics

In addition to logs, the system produces metrics that measure system performance.
Metrics capture aggregated data over time and help engineers identify patterns in system behavior.

Monitoring metrics such as request rate, latency, and error frequency allows teams to detect performance degradation early.



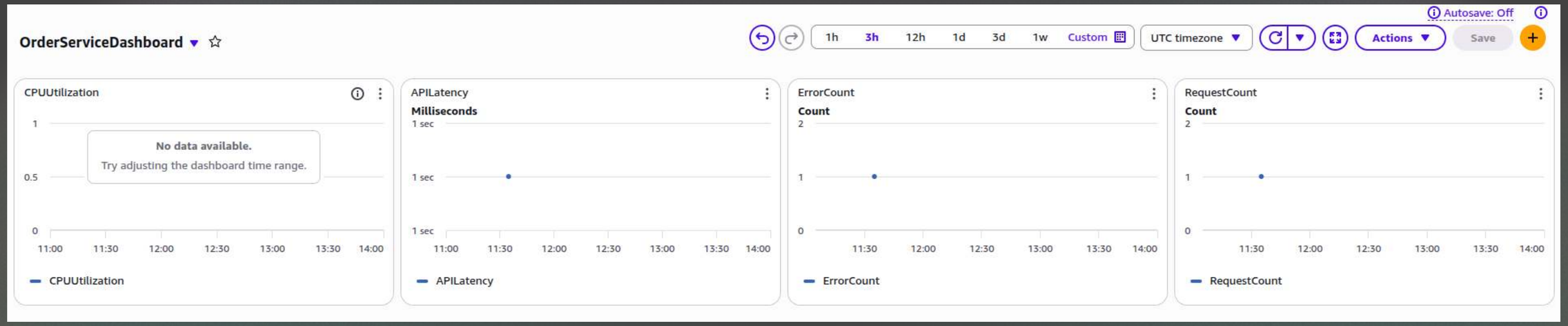
07

Monitoring Dashboard

Dashboards provide a centralized way to visualize system performance.

Instead of analyzing metrics individually, dashboards present key indicators together in a single interface.

This allows engineers to quickly assess system health and detect abnormal trends.



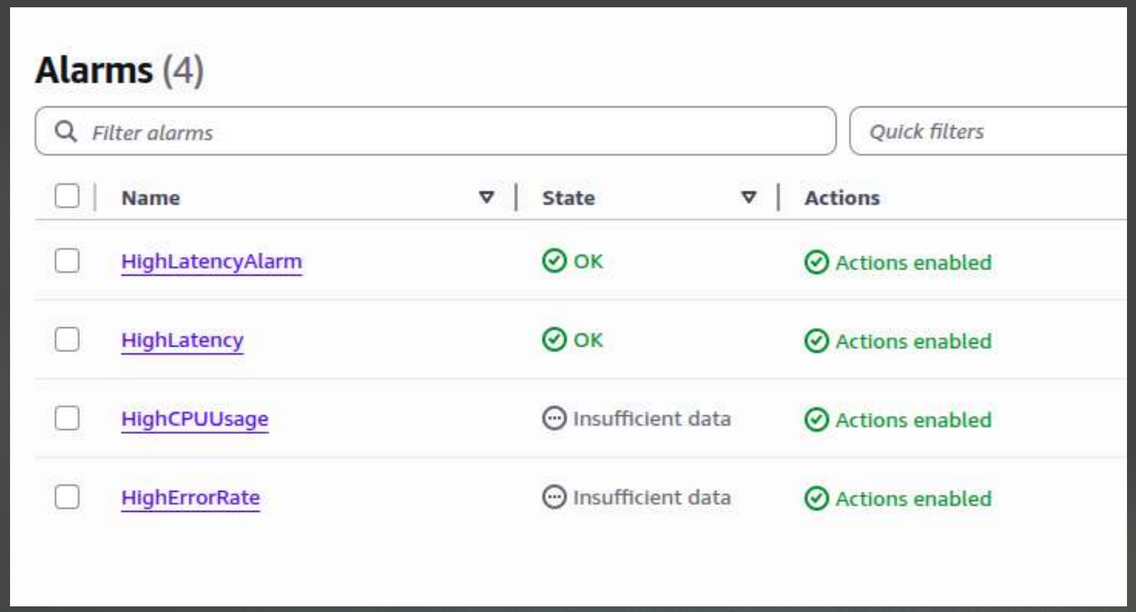
08

Alerting System

An automated alerting system was configured to detect abnormal system behavior.
Alerts are triggered when specific thresholds are exceeded.

Examples include:
- high latency
- high error rate
- high CPU utilization

Alerts are integrated with Amazon SNS to ensure that system issues can be detected and addressed quickly.



09

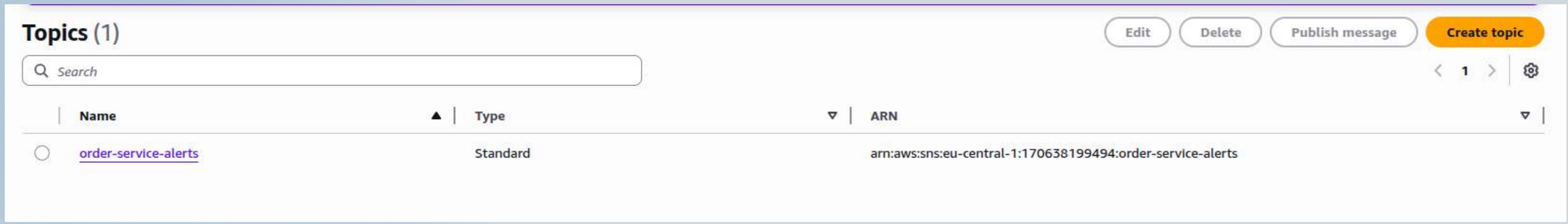
Notification Mechanism

Monitoring systems are most effective when they can automatically notify engineers about abnormal conditions.

In this project, CloudWatch alarms are integrated with Amazon SNS, which allows alerts to be delivered when certain thresholds are exceeded.

How the notification system works

- CloudWatch continuously monitors system metrics
- When a threshold is exceeded, an alarm is triggered
- The alarm sends a notification through SNS
- Engineers receive alerts and can investigate the issue



10

key Learnings

This project highlights how observability improves the reliability and maintainability of cloud systems.

Instead of relying on manual debugging, engineers can use logs, metrics, and alerts to understand system behavior in real time.

Main takeaways:

- Observability should be built into the system from the start
- Structured logging improves debugging and traceability
- Metrics help detect performance issues early
- Dashboards provide visibility into system health
- Alerts enable rapid response to incidents

Overall lesson

Combining logs, metrics, dashboards, and alerts allows engineers to operate systems more confidently and respond faster when problems occur.

11

Challenges

One of the main challenges during this project was making sure that all the observability components were correctly connected. It was not only about deploying the application, but also ensuring that structured logs, custom metrics, dashboards, and alarms were working together as a complete monitoring system.

Another challenge was troubleshooting configuration issues while integrating the application with CloudWatch. Small configuration mistakes could prevent metrics or logs from appearing correctly, so it required careful verification and testing.

Through this process, I developed a better understanding of how different observability signals — logs, metrics, and alerts — complement each other and help provide visibility into the behavior and health of a running service.

12

Future Improvements

If I had more time to continue working on this project, I would probably expand the monitoring setup and add a few more metrics to better understand the behavior of the service.

I would also like to experiment with additional observability tools and explore different ways of visualizing the data in CloudWatch dashboards.

Since this project was mainly focused on learning the fundamentals of observability, another improvement would simply be spending more time testing different scenarios and failures to better understand how the monitoring system reacts.